

Reporte de Estudio: Minería Web, Recuperación de Información y Procesamiento de Textos

1. Fundamentos de la Recuperación de Información (IR)

La Recuperación de Información (IR) se define como la disciplina centrada en obtener datos pertinentes de grandes colecciones, con un énfasis técnico en el tratamiento del texto libre. Desde una perspectiva de arquitectura de datos, es crucial distinguir entre información estructurada y no estructurada. Mientras que el almacenamiento no exige necesariamente una estructura rígida, la transformación de datos no estructurados en formatos de filas y columnas es un **requerimiento técnico obligatorio** antes de ejecutar cualquier proceso de Machine Learning.

Característica	Datos Estructurados	Datos No Estructurados
Definición	Datos con un número fijo de columnas y tipos de datos específicos (esquema rígido).	Datos sin un formato predefinido o rígidamente organizado.
Ejemplos Comunes	RDBMS (SQL), hojas de cálculo de Excel, archivos CSV.	Imágenes, audio, video, formatos de archivos especializados.
Foco del NLP	Generalmente fuera del alcance principal del procesamiento lingüístico.	Texto libre (free-form text).

El texto libre representa el núcleo del NLP porque requiere una orquestación compleja para normalizar la variabilidad del lenguaje humano en vectores numéricos procesables por modelos computacionales.

2. Adquisición de Datos Web y el Proceso ETL

La adquisición de datos constituye la fase de captura de la "materia prima". En arquitecturas modernas, este flujo se integra en el ciclo **Extract, Load, and Transform (ETL)**, adaptado a la escala de la minería de textos:

- **Extract (Extracción):** Obtención de texto desde sistemas de origen, tales como facturas de clientes en PDF o el raspado (*scraping*) de contenido HTML de sitios web.
- **Transform (Transformación):** Consolidación, agregación y limpieza del texto para eliminar el ruido semántico y técnico.
- **Load (Carga):** Almacenamiento en sistemas de destino. Dependiendo de la necesidad de recuperación, se emplean soluciones como **Amazon S3**

(almacenamiento de objetos), **ElasticSearch** o **Redis** (bases de datos de documentos) y **PostgreSQL** (relacionales).

Las fuentes de datos para NLP se dividen en tres categorías estratégicas:

- **Datos Internos:** Históricos de la organización (transcripciones de llamadas, feedback).
- **Sitios Web y APIs:** Datos suplementarios obtenidos mediante servicios web.
- **Datos de Usuario:** Flujos constantes generados en tiempo real por la interacción en apps y plataformas.

3. Web Data Mining: Servicios Web, APIs y HTTP

Un servicio web es una aplicación alojada en un servidor que proporciona datos mediante el modelo **cliente-servidor**. El estándar arquitectónico predominante es **REST** (Representational State Transfer). La comunicación se rige por el protocolo HTTP, donde el servidor devuelve códigos de estado que definen el resultado de la transacción. Siguiendo las definiciones literales del estándar:

- **200: "OK"** - The request succeeded and the response was returned successfully.
- **404: "Not Found"** - There is nothing at the path where the request was made (resource does not exist).
- **503: "Unavailable"** - The server is unable to return a response to the request (usually because the server is down).

4. EJEMPLO PRÁCTICO 1: Extracción de HTML con la Librería **requests**

Para un especialista, la minería web efectiva es un proceso de dos pasos: primero, obtener el código fuente mediante **requests** y, segundo, extraer datos estructurados utilizando herramientas como **BeautifulSoup**. El siguiente bloque muestra el paso inicial de adquisición.

```
import requests
```

```
# Realizar una petición GET programática
```

```
url = "https://www.google.com"
```

```
r = requests.get(url)
```

```
# Verificación del código de estado (Response [200])
```

```
print(f"Código de estado: {r.status_code}")
```

```
# Extracción del texto plano (primeros 500 caracteres)
```

```
# Este contenido se pasaría posteriormente a BeautifulSoup para su parseo
```

```
print("Fragmento del contenido HTML:")
```

```
print(r.text[0:500])
```

5. Minería de Textos: Preprocesamiento y Tokenización

El preprocesamiento garantiza la **consistencia** técnica en todo el dataset. Un **documento** es la unidad mínima de texto (ej. un tuit), mientras que el **corpus** es el conjunto total de documentos.

BIAS ALERT: Es fundamental reconocer que la vasta mayoría del ecosistema NLP actual está diseñado para el idioma inglés. El procesamiento de otros idiomas suele categorizarse como NLP multilingüe y requiere consideraciones específicas de normalización.

La **tokenización** descompone el texto en unidades constituyentes (tokens). Para asegurar la estabilidad del corpus, se utilizan librerías como **NLTK** y **Scikit-learn**, que permiten transformar la data no estructurada en una estructura apta para el entrenamiento de modelos.

6. EJEMPLO PRÁCTICO 2: Tokenización y Limpieza de Ruido

La eliminación de ruido (puntuación y espacios) y la estandarización de caso son pasos obligatorios para reducir la varianza en el modelo.

```
import nltk
from nltk.tokenize import word_tokenize
import string

# Necesario para entornos controlados como Google Colab
nltk.download('punkt')

text = "La Minería Web requiere consistencia técnica. ¿No es así?"

# Estandarización a minúsculas (evita duplicidad de tokens como 'La' y 'la')
text_lower = text.lower()

# Tokenización y filtrado de puntuación
tokens = word_tokenize(text_lower)
cleaned_tokens = [t for t in tokens if t not in string.punctuation]

print(f"Tokens normalizados: {cleaned_tokens}")
# Salida esperada: ['la', 'minería', 'web', 'requiere', 'consistencia', 'técnica', 'no', 'es', 'así']
```

7. Normalización Avanzada: Stemming y Lematización

Stemming

Es una técnica basada en reglas heurísticas (algoritmos de Porter o Snowball) que recorta sufijos para hallar la raíz. Sin embargo, carece de precisión lingüística; por ejemplo, una regla que elimine "-ing" podría convertir erróneamente **"running"** en **"runn"**, generando un token inexistente en el diccionario.

Lematización

Utiliza un vocabulario y análisis morfológico complejo para identificar el **lema** (forma base real). Es un proceso superior al stemming ya que garantiza que el resultado sea una palabra válida, aunque demanda mayor capacidad de cómputo.

8. EJEMPLO PRÁCTICO 3: Reducción Morfológica (Stemming)

A continuación, se observa el comportamiento del algoritmo de Porter ante formas verbales.

```
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
words = ["running", "runs", "ran"]
```

```
stems = [ps.stem(w) for w in words]
```

```
print(f"Palabras: {words}")
```

```
print(f"Stems: {stems}")
```

```
# Salida esperada: ['run', 'run', 'ran']
```

```
# Nota: 'ran' permanece igual por ser una forma irregular no cubierta por reglas simples.
```

9. Estructuración de Datos: Vectorización y Matrices Documento-Término (DTM)

La vectorización convierte los tokens en características numéricas, organizándolos usualmente en una **Matriz Documento-Término (DTM)**:

1. **Filas:** Representan cada **documento** individual del corpus.
2. **Columnas:** Representan los **términos** (tokens) únicos que actúan como características (*features*).
3. **Enfoques de Valorización:**
 - **Count Vectorization:** Conteo de frecuencia absoluta.
 - **TF-IDF:** Pesa la importancia del término inversamente a su frecuencia en el corpus.
 - **Embeddings:** Representaciones estadísticas de vanguardia que capturan semántica profunda. Representan el **estado del arte**, pero su implementación conlleva una **mayor complejidad arquitectónica y computacional**.

10. Ética y Calidad: El Sesgo Algorítmico en la Minería de Datos

Consideraciones Críticas: El sesgo algorítmico no es una falla del código *per se*, sino un reflejo de los datos de entrenamiento recolectados. En minería web, si los datos están sesgados históricamente, el sistema entrará en un **bucle de feedback** (*feedback loop*) donde el algoritmo aprende y perpetúa continuamente patrones injustos. Esto genera un **impacto dispar** (*disparate impact*), afectando desproporcionadamente a grupos protegidos en áreas sensibles como la salud o la justicia. El incumplimiento ético no solo erosiona la confianza del usuario, sino que acarrea riesgos legales severos, como las multas estipuladas en la Ley de IA de la UE (hasta el 7% del volumen de negocios anual).

11. Conclusión y Bibliografía

El éxito en la extracción de conocimiento desde la web reside en un flujo de trabajo riguroso: **Adquisición** (vía servicios web), **Preprocesamiento** (asegurando consistencia y eliminando ruido) y **Vectorización** (estructuración numérica). La integración de herramientas como `requests`, `BeautifulSoup` y `NLTK` es fundamental para transformar datos no estructurados en activos estratégicos, siempre bajo un marco de gobernanza que mitigue el sesgo y el impacto dispar en los resultados.